Angewandte Kryptographie

Florian Handke
Campus Schwarzwald

Agenda

1. Root of Trust
2. Code Signing
3. Secure **Boot**
4. Secure **Update**
5. Secure **Device Identifiers (Certificates)**

Root of Trust

Establishes the **foundational security element** that ensures all subsequent processes in a device's lifecycle are trustworthy.

Root-of-Trust (RoT)

... as standardized by the Trusted Computing Group

Source: Infineon

Trusted Platform Module (TPM)

A physical hardware module for cryptographic operations and secure storage.

TRUSTED® COMPUTING GROUP

Secure Element

A Secure Element is a tamper-resistant component used to securely store and manage sensitive information such as cryptographic keys, authentication credentials, and other sensitive data

Source: STMicro

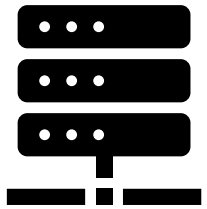
Hardware Security Modules (HSMs)

A dedicated hardware devices used to manage cryptographic keys and perform secure cryptographic operations.

Source: Thales

How Hardware Security Modules Work

1 Root of Trust



1 Request a cryptographic operation (e.g. sign a digital certificate)

3 Provide the resulting output (e.g. a signed certificate)

Hardware Security Module (HSM)

AWS CloudHSM

Source: aws

Source: Nitrokey

Source: Thales

Source: Utimaco

2

Securly sign the certificate using a private key within the HSM's isolated environment

Code Signing

Process of **digitally signing software or scripts** to verify their authenticity, integrity, and origin, ensuring they haven't been altered or tampered with since signing.

Why we sign code

Authenticity

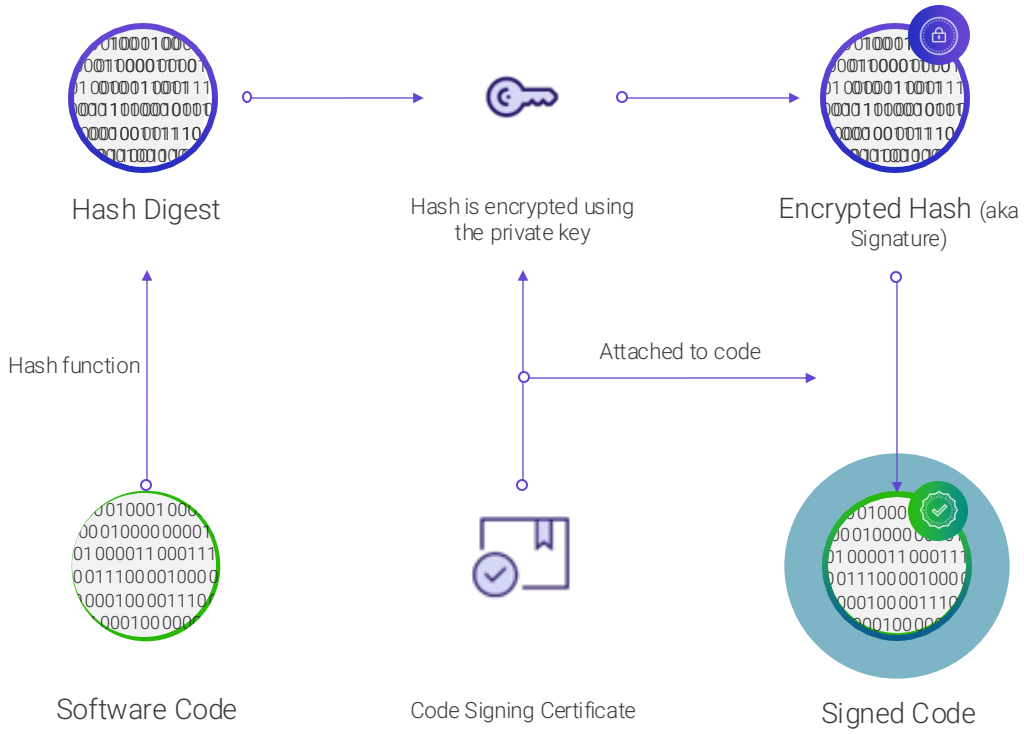
Like a written signature, it provides valid proof of who the author of the code is.

Integrity

It also ensures that the code hasn't been tampered with after it was signed.

Trust

Overall, provides end-users with trust that the software is authentic and unaltered



And why not a Checksum!?

Checksum can detect *accidental* changes in data
Checksum can't protect against *malicious* changes

Execution of software requires deployment of a chain of trust

1

SW is developed

In a trusted (IT) environment, applying secure coding rules & using verification tools integrating libraries checked against known vulnerabilities

2

SW is digitally signed

Firmware and applications images are signed with versioning, using **OEM private key securely stored** and securely used by SW issuer (e.g. OEM)

3

Verification key(s) is (are) installed

The right verification credentials must be installed and "locked" on the product in mass production

4

SW verification is enforced

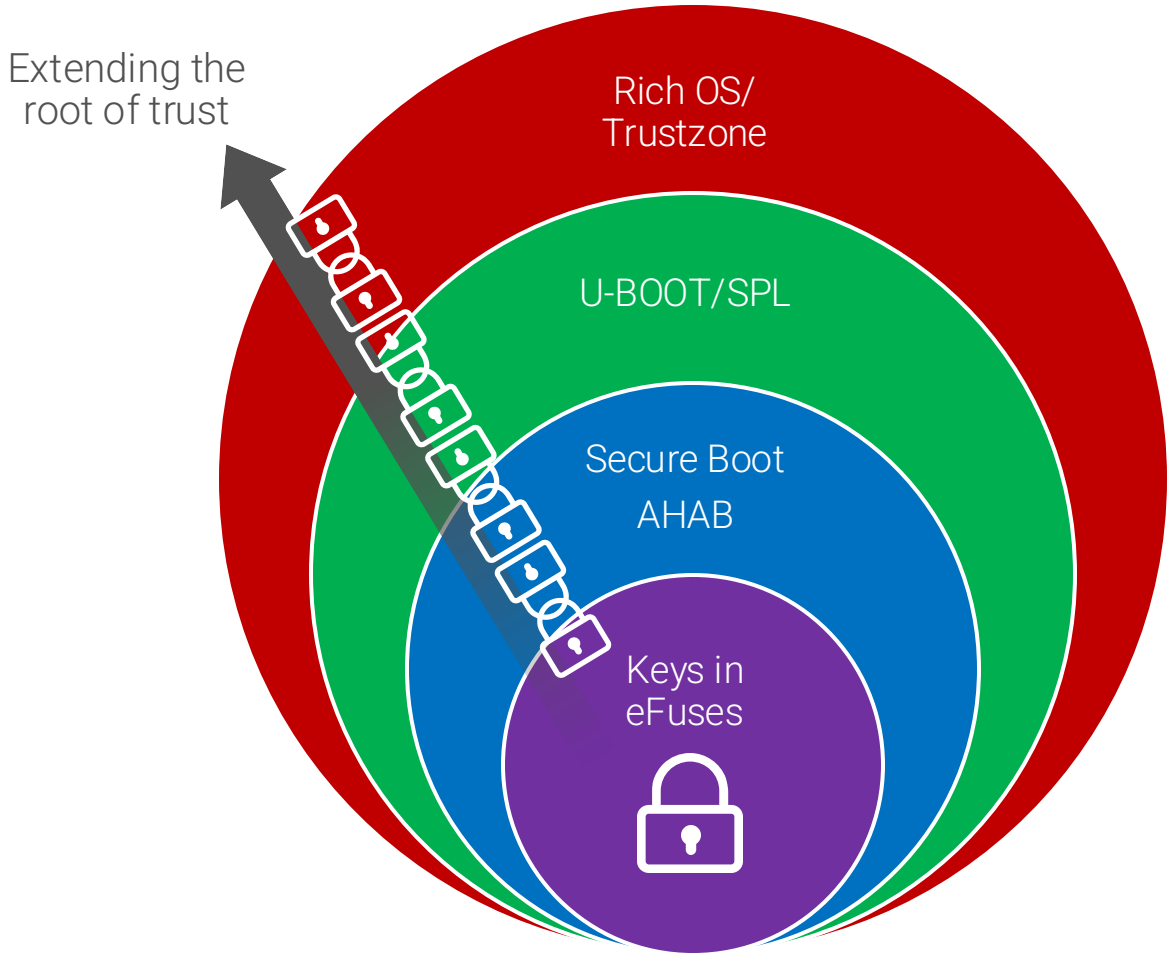
Firmware and applications images are verified

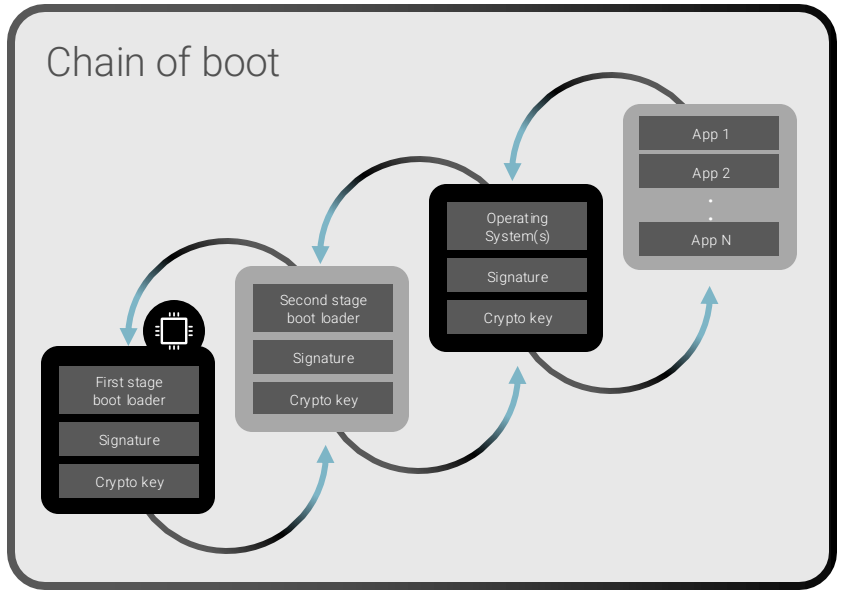
- By a **trusted hardware**
- With a **public key protected** in integrity on the device (stored in immutable memory) and with a monotonic version counter

Secure Boot

Uses the root of trust to verify the **integrity and authenticity of the firmware** during startup, preventing unauthorized code from executing.

A multistage process

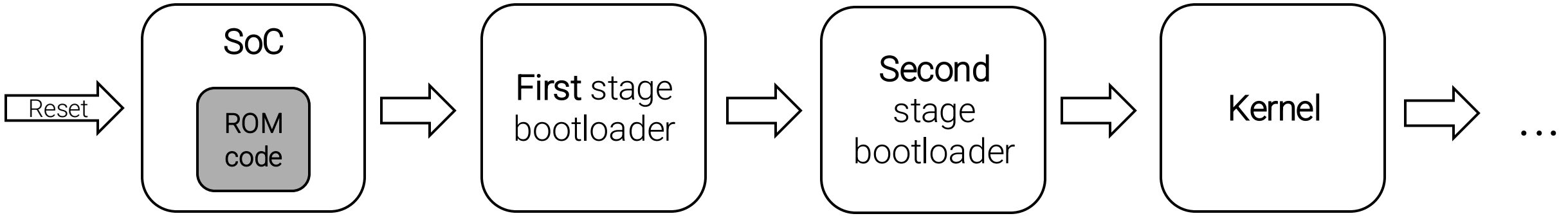




Extracts from: https://www.european-cyber-resilience-act.com/Cyber_Resilience_Act_Annex_1.html

Secure boot for constrained devices

Process



Initializes basic hardware and loads the first stage bootloader.

Performs **minimal setup** and loads the second stage bootloader.

Completes **hardware initialization** and loads the kernel.

Initializes the operating system and transitions the system to operational mode.

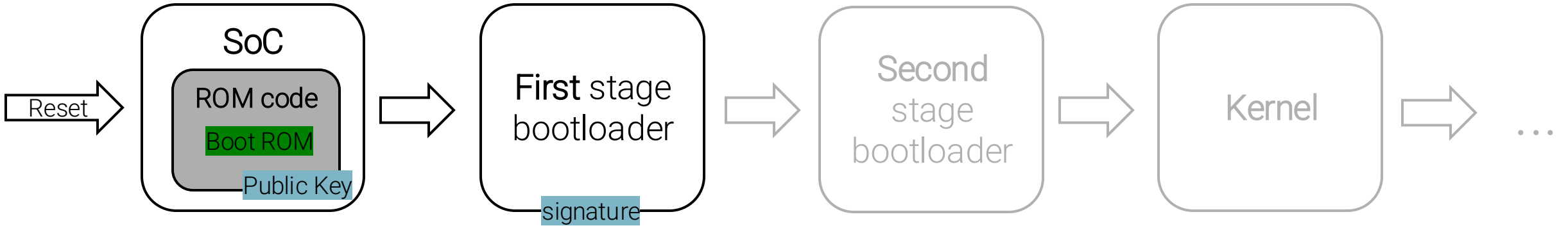
Secure boot for constrained devices

Options

Component	Options Available
System on Chip (SoC)	- **Broadcom BCM** (e.g., BCM2711 for Raspberry Pi) - **Espressif ESP** (e.g., ESP32, ESP8266) - **STMicroelectronics STM32** (e.g., STM32F4, STM32H7) ...
First Stage Bootloader	- **X-Loader** (MLO) - **SPL (Secondary Program Loader)** in U-Boot ...
Second Stage Bootloader	- **U-Boot** (Universal Bootloader) - **Barebox** - **GRUB (GRand Unified Bootloader)** ...
Kernel	- **Linux Kernel** - **FreeRTOS** - **Zephyr** ...

Secure boot for constrained devices

Process (1/4)



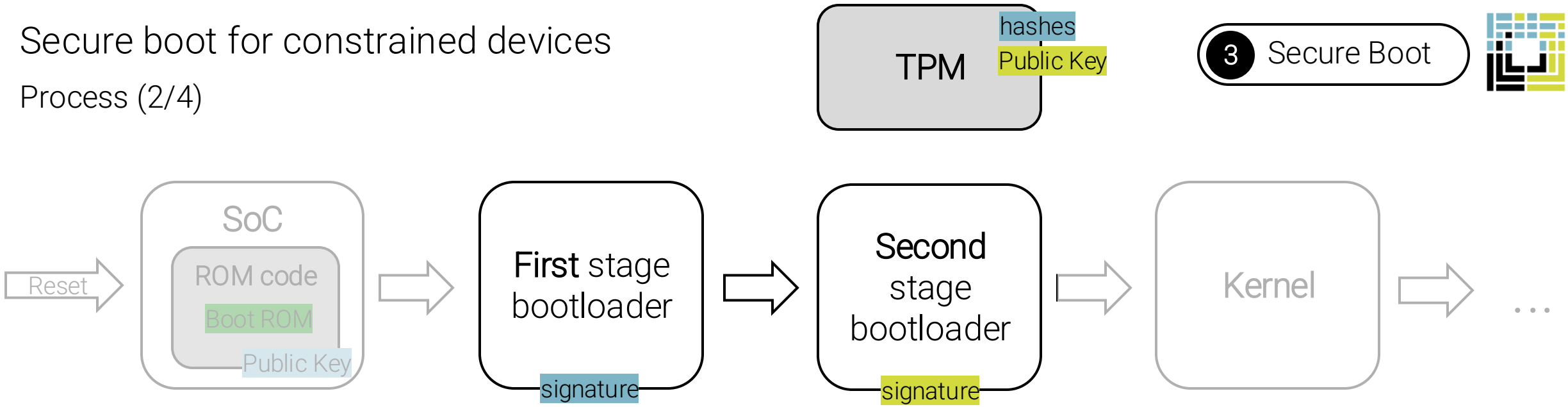
Power-On/Reset
The boot process begins when the device is powered on or reset. The SoC initializes essential hardware components, including the CPU, memory controller, and clock settings.

Boot ROM Execution
The SoC's built-in Boot ROM code (also known as BootROM) is executed. This code is usually stored in a secure, non-modifiable memory region and is the first code that runs on the device. It performs basic system checks and verifies the authenticity of the first stage bootloader before loading it.

Signature Verification
The BootROM typically includes cryptographic verification, where it checks the digital signature of the first stage bootloader. This ensures that only bootloaders signed with a trusted private key (whose corresponding public key is stored in SoC ROM) can be executed.

Secure boot for constrained devices

Process (2/4)



Minimal Initialization with Security Checks

The first stage bootloader (e.g., X-Loader, SPL) is a small, lightweight piece of code loaded into memory by the SoC after passing signature verification.

TPM Integration

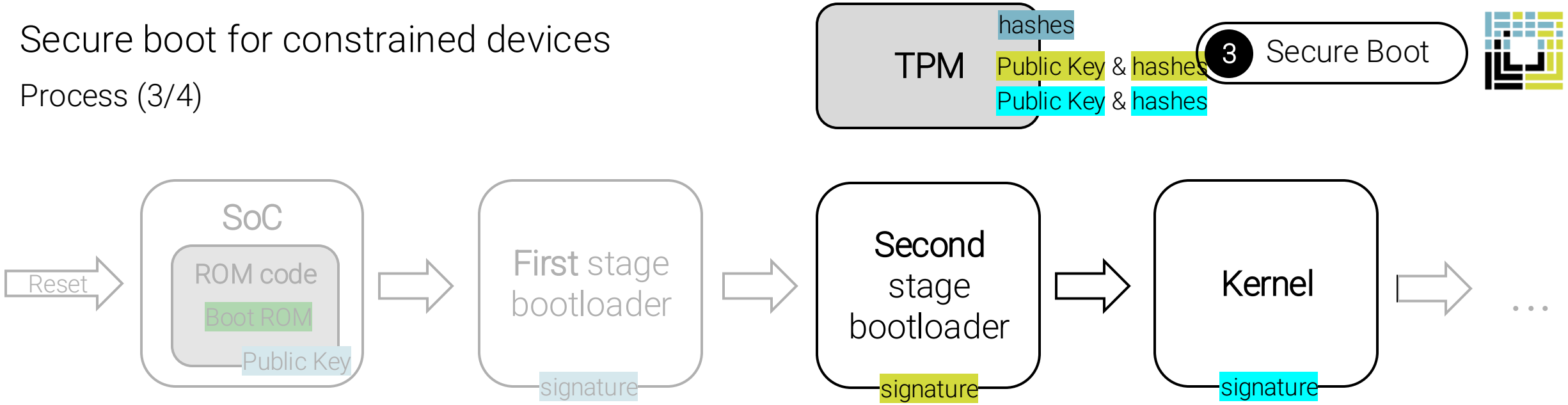
If the system includes a Trusted Platform Module (TPM), the first stage bootloader can use the TPM to store cryptographic keys and perform secure operations. The TPM can also measure the integrity of the boot process by taking hashes of the bootloader and other critical components and storing these measurements securely.

Loading Second Stage Bootloader

After the minimal setup, the first stage bootloader verifies the signature of the second stage bootloader (using the same or another set of cryptographic keys) and loads it into memory if the signature is valid.

Secure boot for constrained devices

Process (3/4)

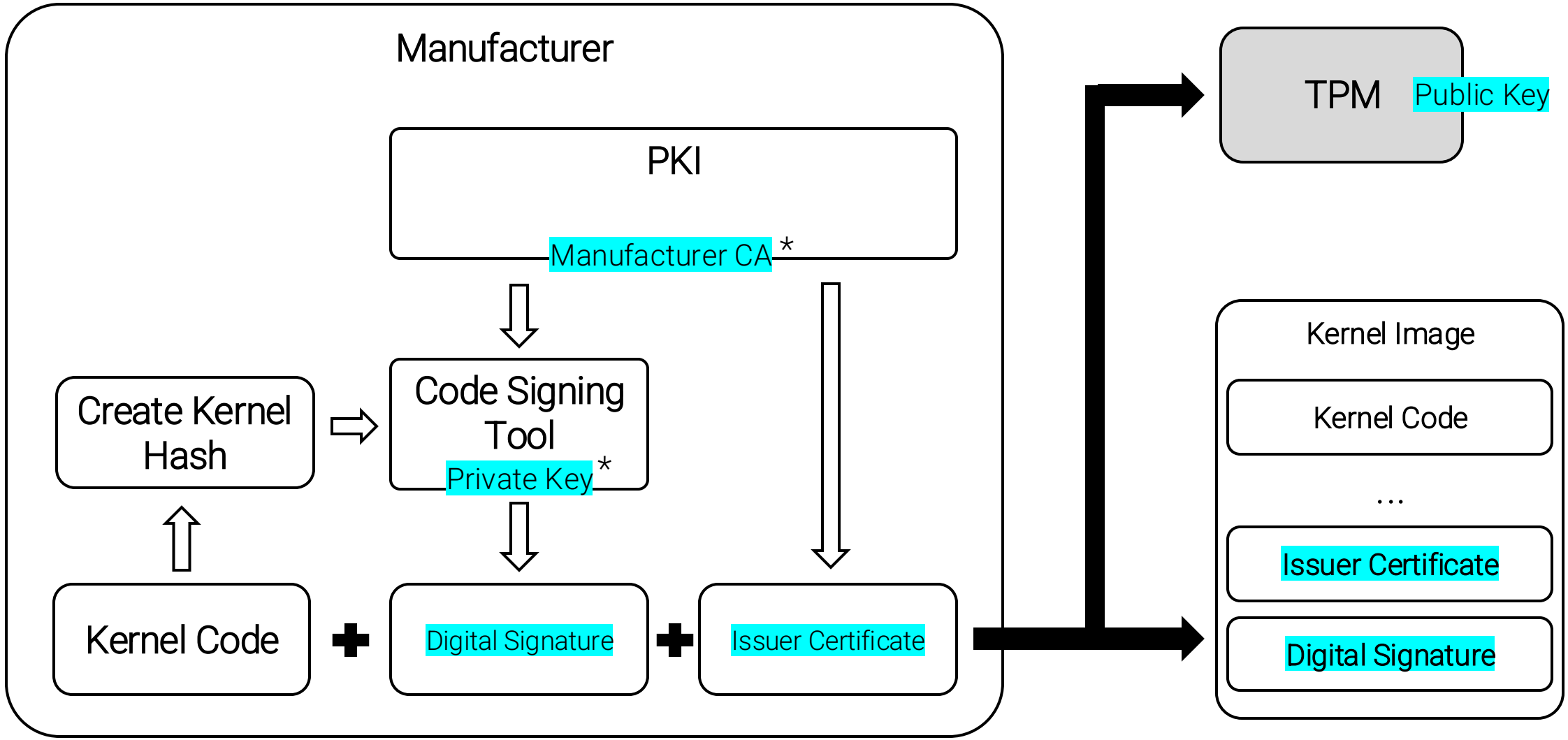


Comprehensive Initialization
The second stage bootloader (e.g., U-Boot) takes over, performing more extensive hardware initialization, including setting up peripherals and network interfaces.

Chain of Trust
The second stage bootloader continues the chain of trust by verifying the signature of the operating system kernel or other components it loads. Each component's integrity is checked before execution to ensure that it hasn't been tampered with.

TPM Measurements
The bootloader may interact with the TPM to store additional measurements (hashes of the kernel and configuration files). These measurements can be used later for attestation, proving that the system booted with a trusted configuration.

Image signing

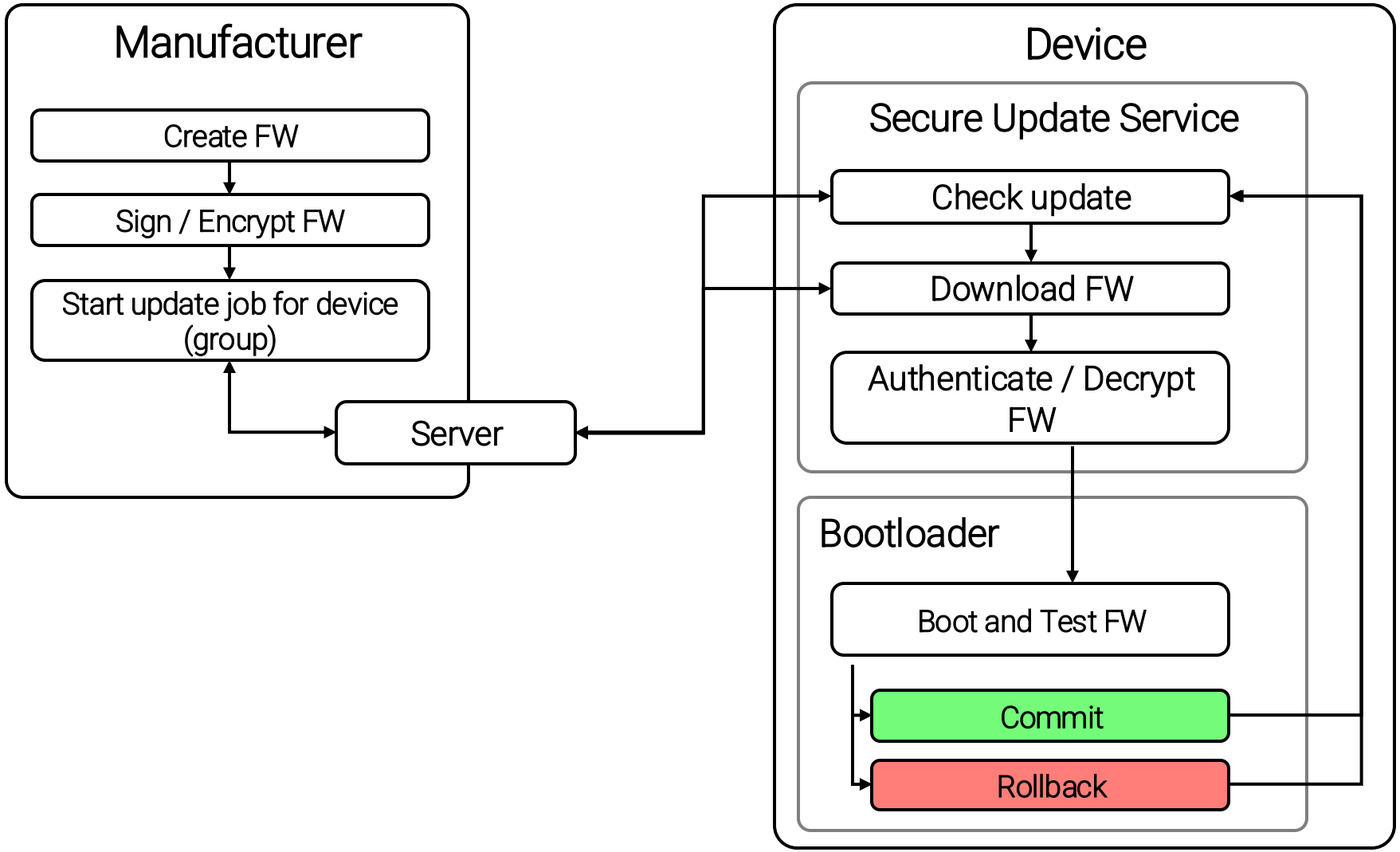


* Storing private keys in an HSM is a best practice

Secure Update

Ensures that only authenticated and verified updates are applied, maintaining the system’s integrity throughout its operational life.

Secure update



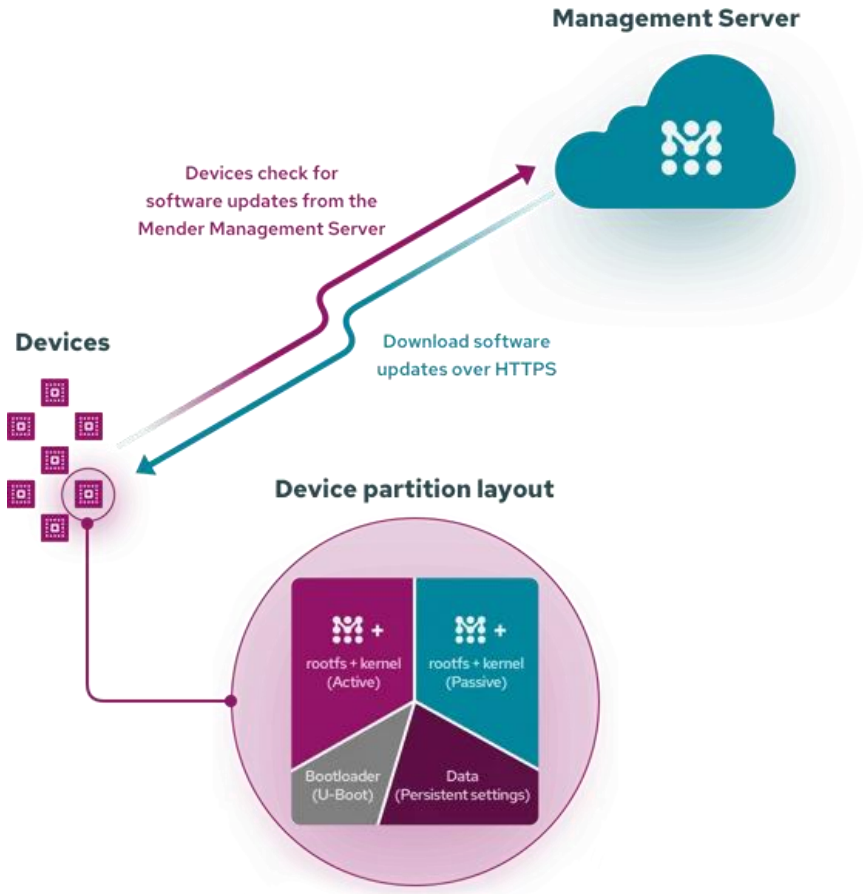
Secure Updates

Tools

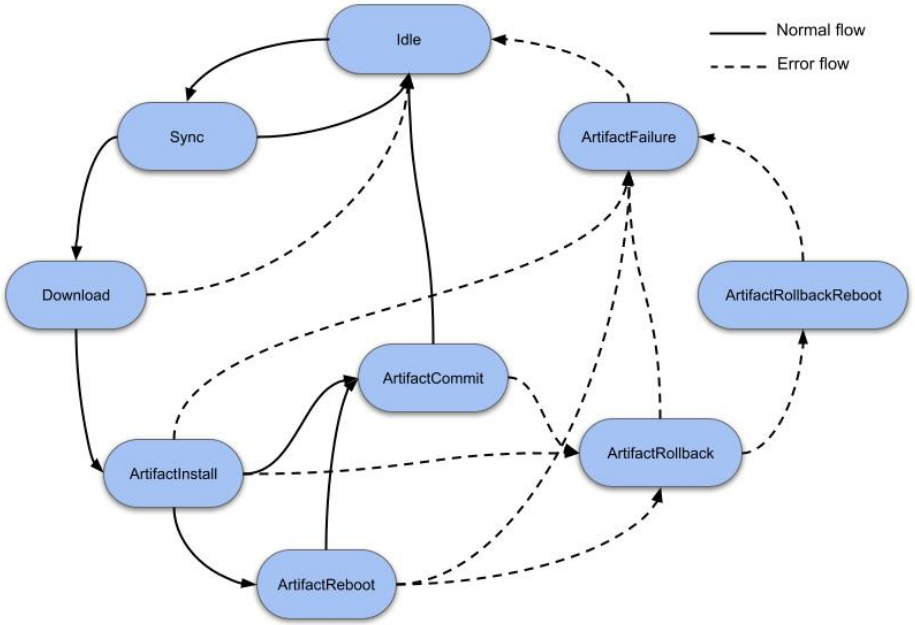
Update Method	Tools
OTA (Over-the-Air)	- Mender - Balena - Hawkbit - SWUpdate - NervesHub - TI SimpleLink OTA - Azure IoT Hub - AWS IoT Device Management - Torizon OTA - QtOTA
Network-Based	- Fleet Commander - Ansible - Red Hat Satellite - Canonical Landscape - Puppet - Chef - Azure IoT Hub - AWS IoT Device Management - Jenkins - WSUS (Windows Server Update Services) - Munki
Manual	- Rufus - YUM/DNF - Apt-get/Aptitude - Etcher - Debian Package Updater

Secure Update

Example with mender



source: mender



source: mender

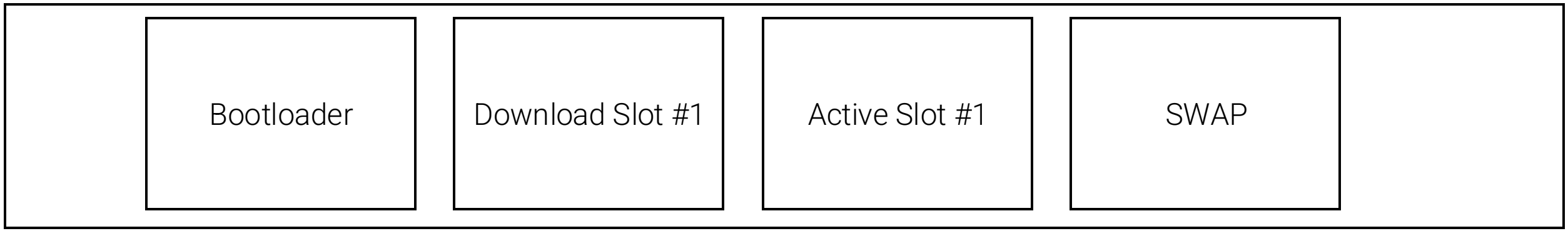
Secure Update

Types of Update

- Operating System (OS)
- Firmware
- Drivers
- Applications and Software Packages
- Security Updates
- Bootloaders
- Digital Certificates and Keys
- Storage and File System
- ...

Secure Update

Firmware – Simple example



Responsible for initializing the hardware, selecting the correct firmware to boot and verifying the firmware

Memory area where the **currently running firmware** is stored

Separate memory area designated for **storing the new firmware** version during an update

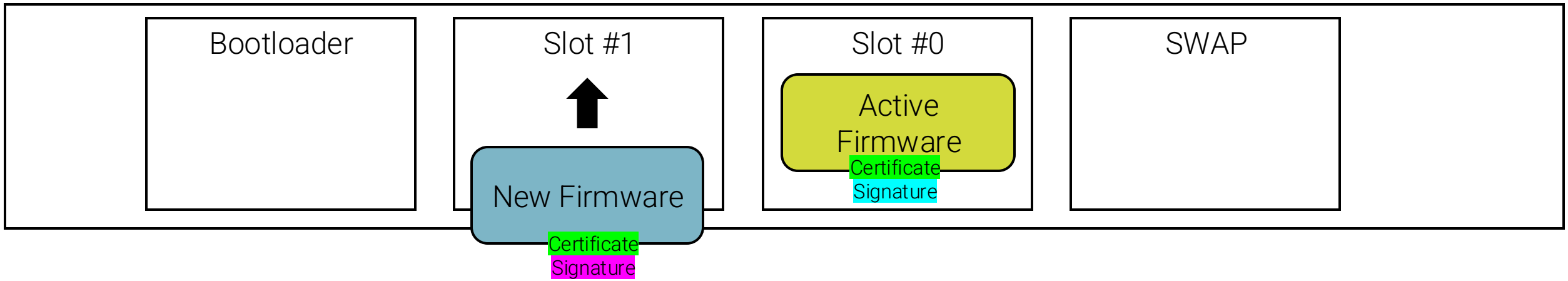
Mechanism refers to the process by which the bootloader **switches the roles of the Active Slot and the Download Slot** after a successful update

Secure Update

Firmware – Simple example

TPM

Public Key



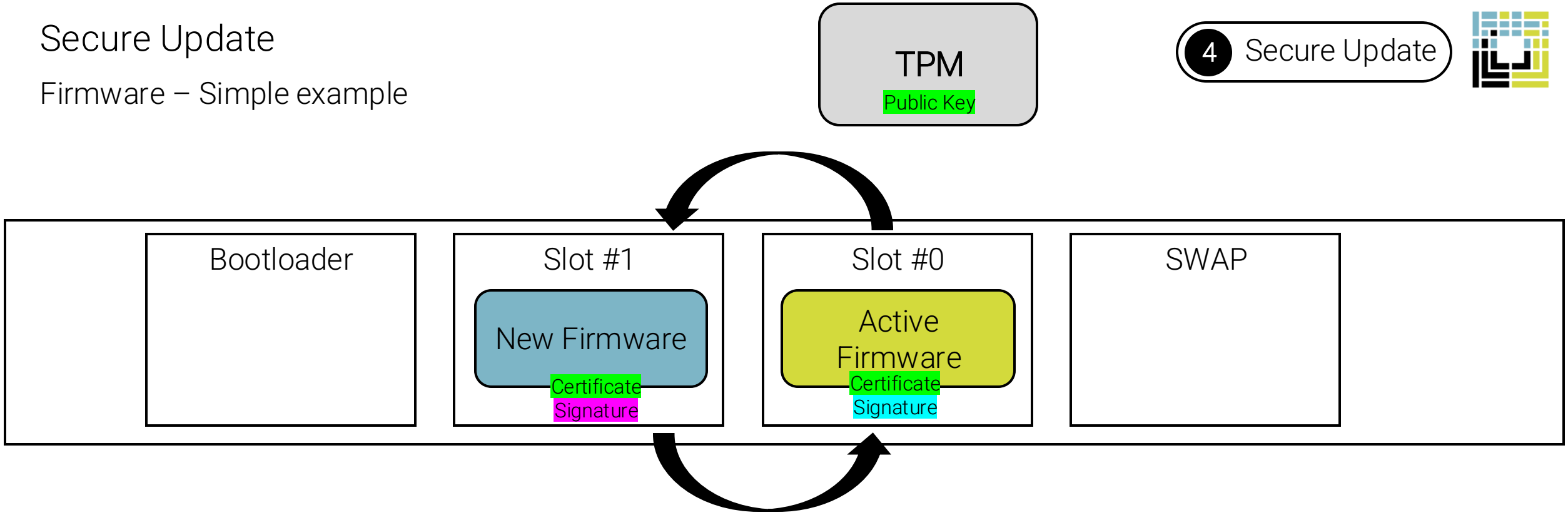
Initiation of the Update
The firmware update process is typically initiated by a user command, a scheduled update, or a trigger from a remote server (e.g., over-the-air (OTA) updates).

Download to Slot #1
The new firmware is downloaded and stored in Download Slot #1.

Verification
After the new firmware is downloaded, the bootloader verifies the integrity and authenticity of the new firmware by checking cryptographic signatures and calculating checksums or hashes.

Secure Update

Firmware – Simple example



Preparation for SWAP
If the verification is successful, the device is prepared for a SWAP operation. This preparation includes marking the new firmware in Download Slot #1 as ready to be swapped.

Reboot
The device is rebooted. During the reboot process, the bootloader checks the status of the firmware slots.

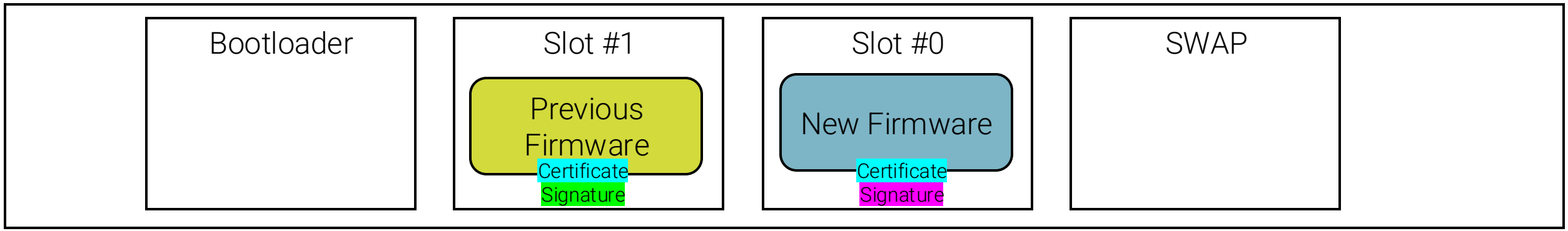
SWAP Operation
The bootloader swaps the pointers or memory mapping so that Download Slot #1 becomes the new Active Slot #1. The previous Active Slot #1 is then marked as the new Download Slot for future updates.

Secure Update

Firmware – Simple example

TPM

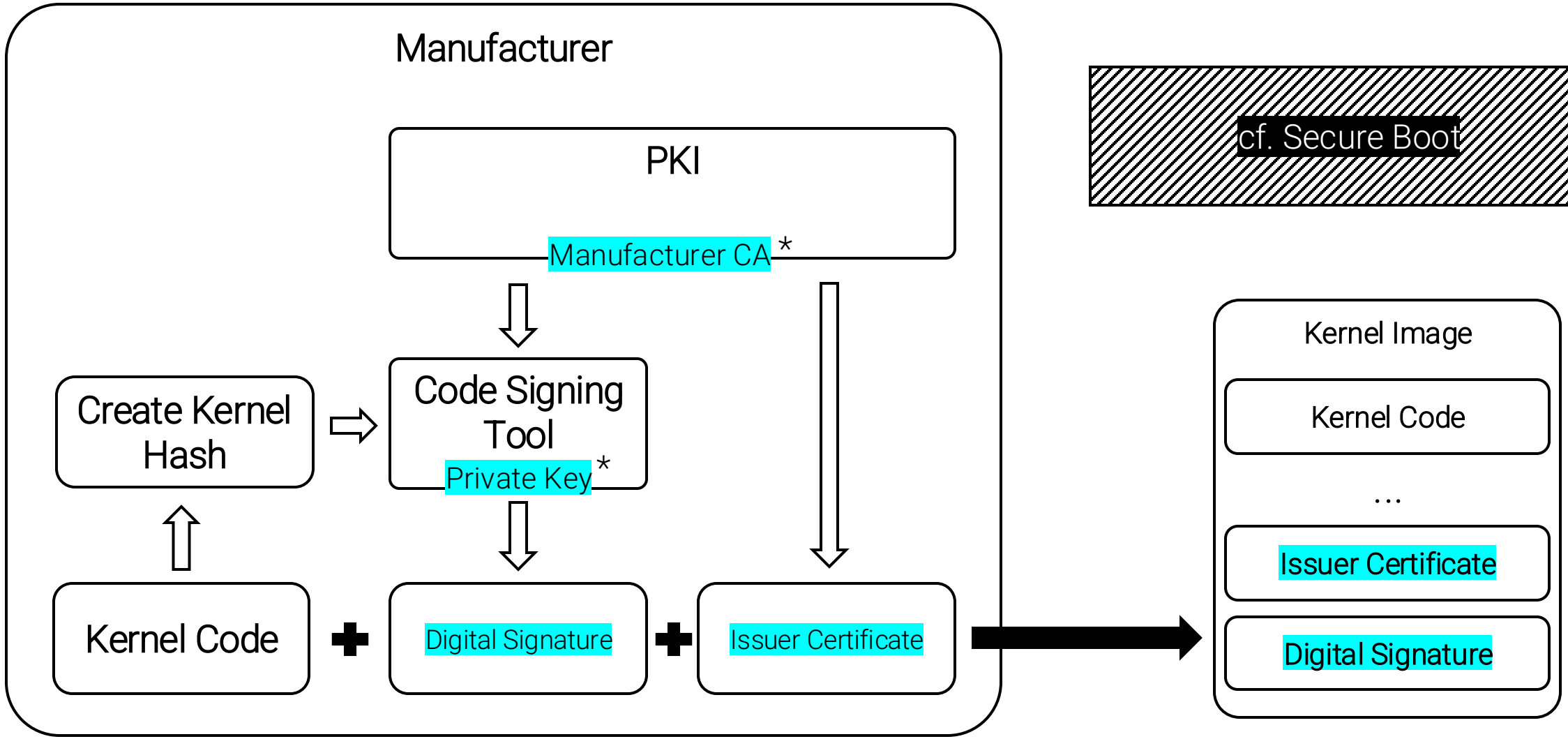
Public Key



Verification Post-SWAP
After the SWAP operation, the bootloader verifies that the new firmware is functioning correctly.

Rollback Machanism
If the new firmware fails to boot or if any issues are detected during the initial operation, the bootloader can revert to the previous firmware in the original Active Slot

Image signing



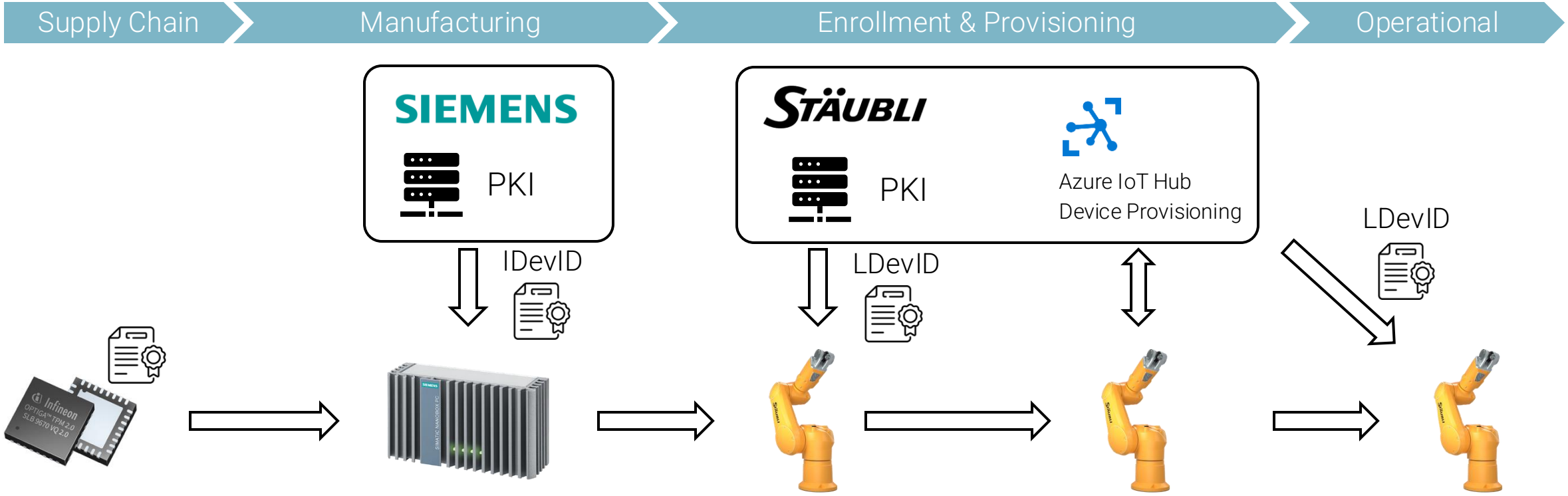
* Storing private keys in an HSM is a best practice

Secure Device Identities

Provides **unique and secure identification of devices** within a network, enabling robust authentication and preventing spoofing or unauthorized access.

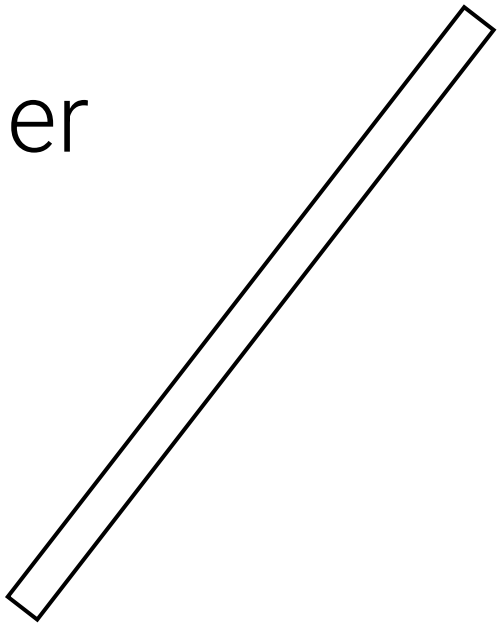
Device Identities Life Cycle

Fictional Scenario



Permanent Identifier and Credential for the Device

- Globally **unique**
- Provisioned by the **device manufacturer**
- Long term authentication and attestation



Domain specific Identifier(s) and Credential(s)

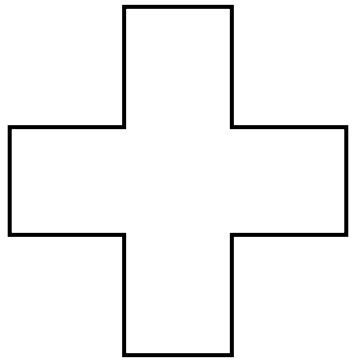
- flexible, configurable identity
- for **local networks**
- managed, rotated, or revoked

IDevID

Initial Device Identifier

LDevID

Locally Significant Device Identifier

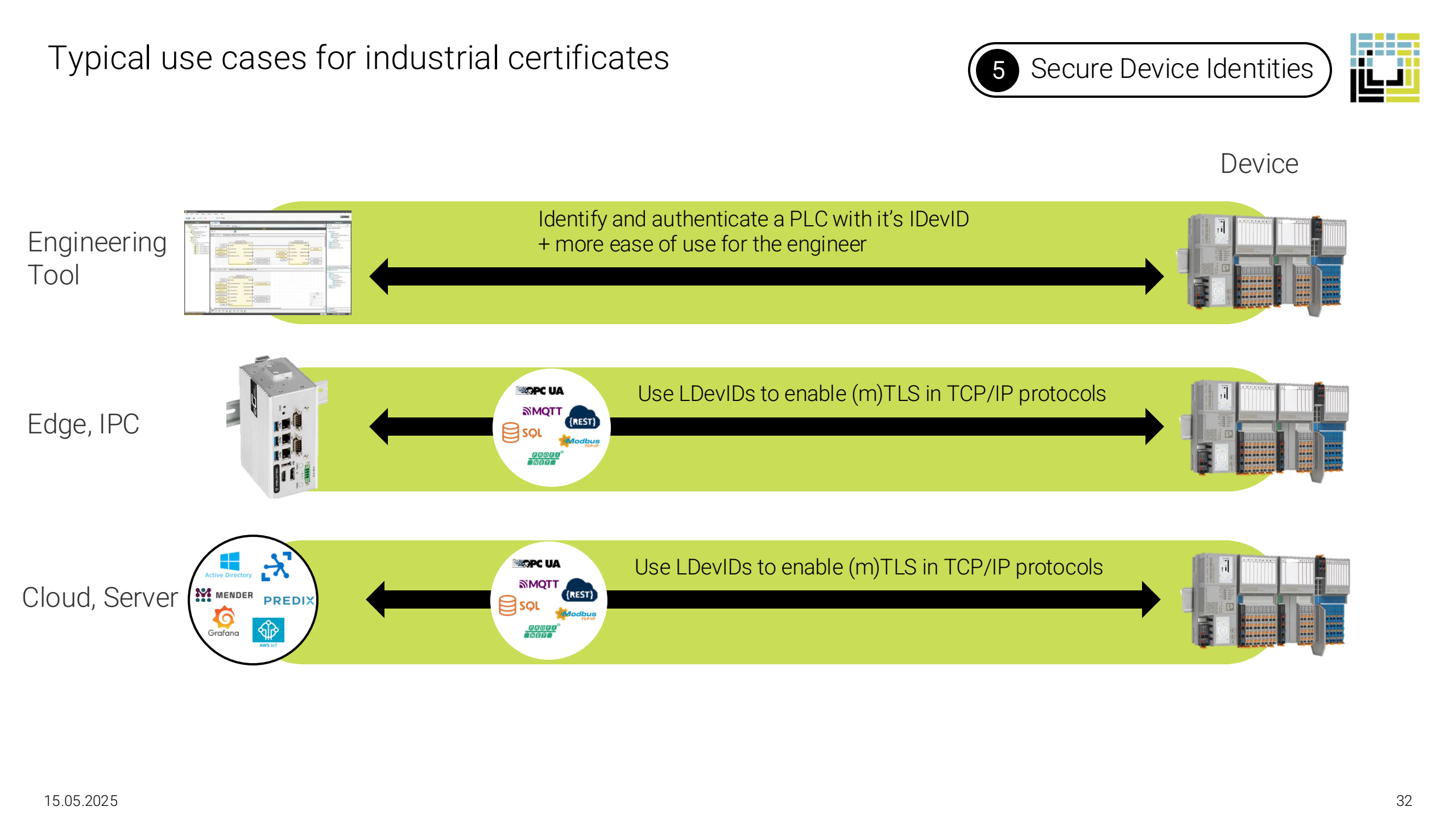


DevID modules

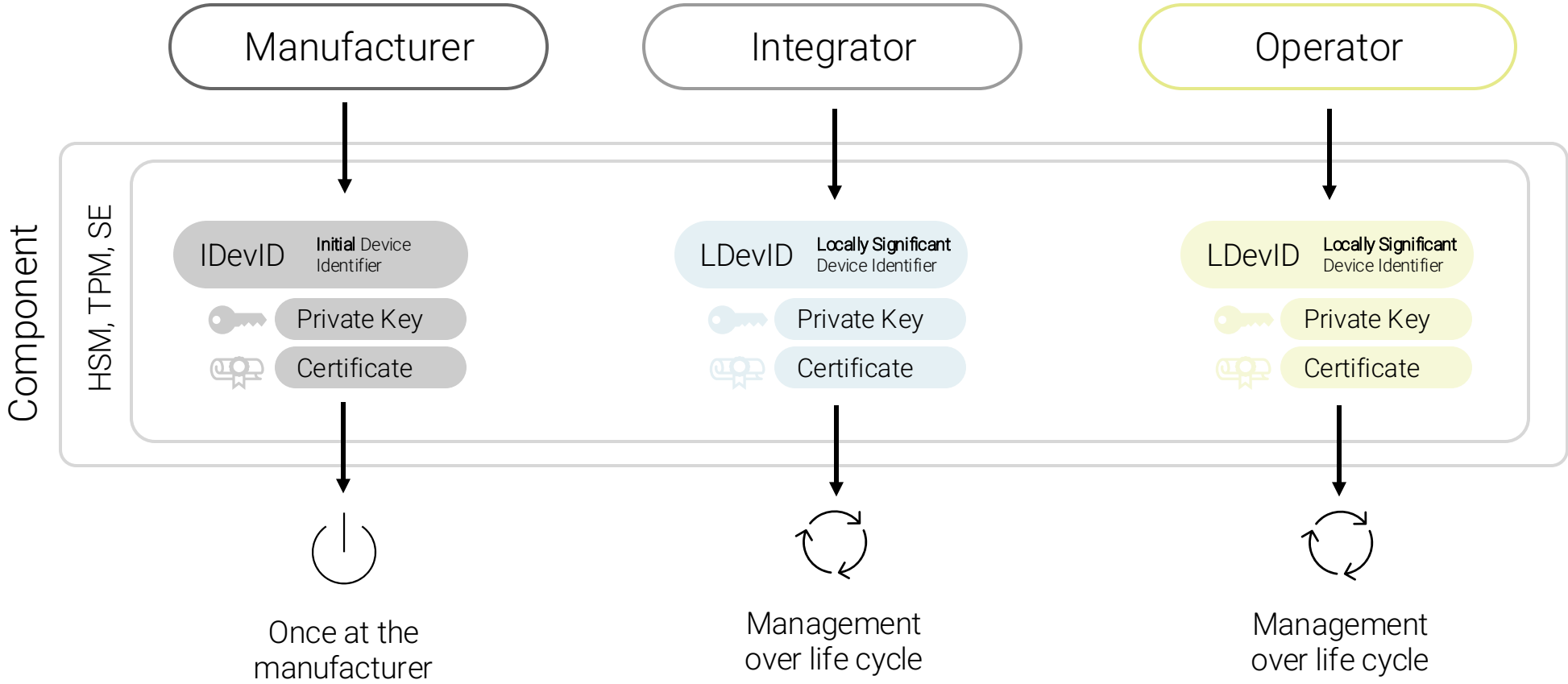
Management capabilities on the device

- service and management interface,
- storage,
- asymetric cryptography
- Random number generators

Typical use cases for industrial certificates

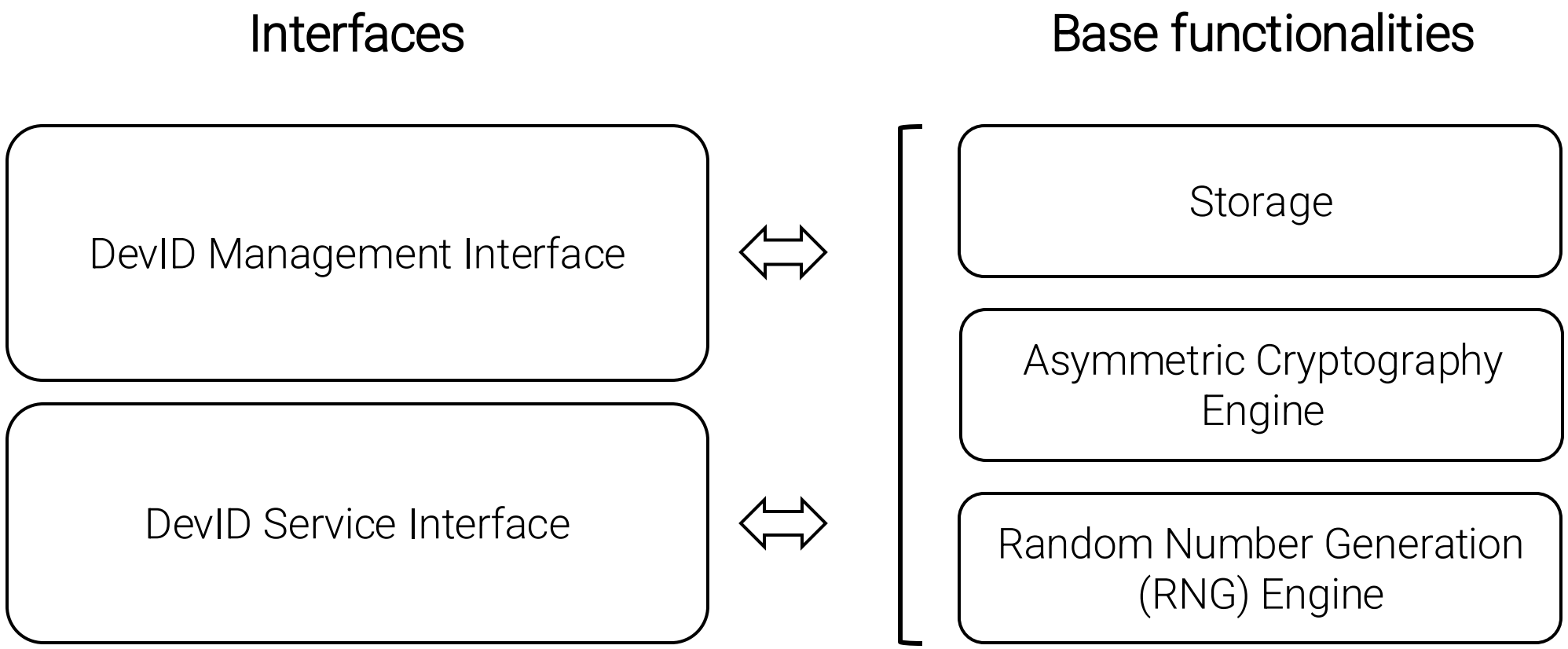


Identification using IEEE 802.1 AR-compliant identities



DevId Modules

Overview



Secure Device Identifiers

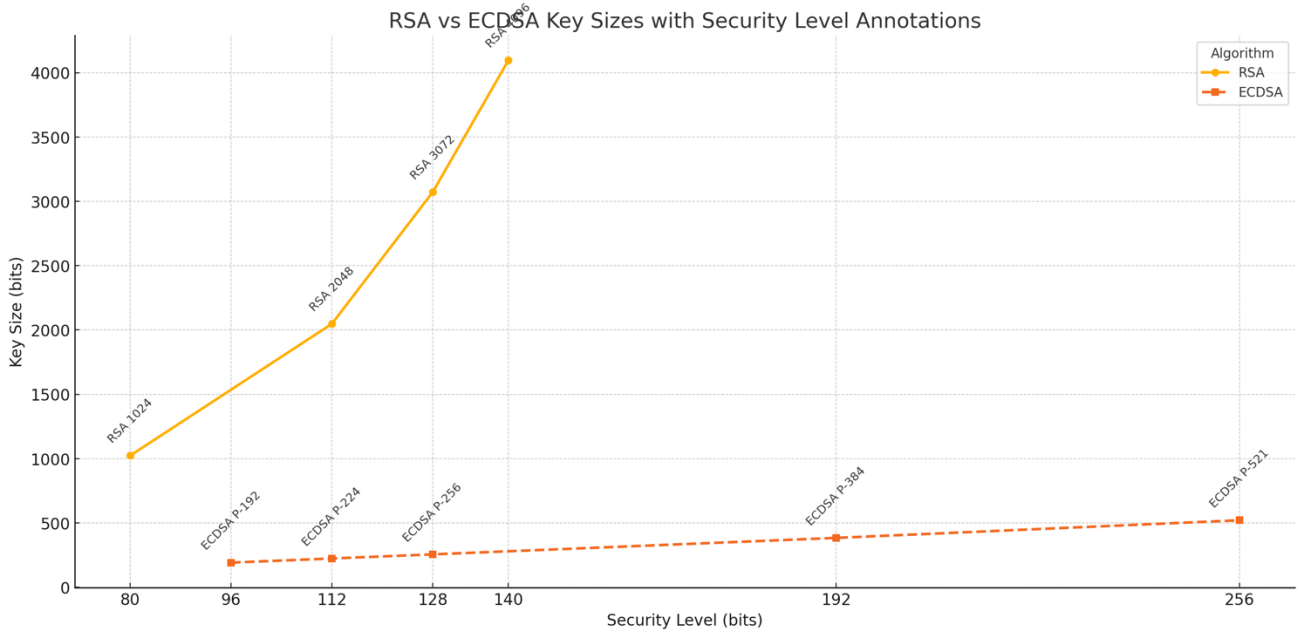
Key and certificate size & key generation time

Part of Certificate	Typical Size for RSA 3072-bit	Typical Size for ECDSA P-256
Version	1-2 bytes	1-2 bytes
Serial Number	4-16 bytes	4-16 bytes
Signature Algorithm	10-20 bytes	10-20 bytes
Issuer	50-100 bytes	50-100 bytes
Validity	20-40 bytes	20-40 bytes
Subject	50-100 bytes	50-100 bytes
Subject Public Key Info	400-450 bytes → Key size: 3072 bits (384 bytes)	70-90 bytes → Key size: 256 bits (64 bytes)
Issuer Unique Identifier	Varies, typically 0-20 bytes	Varies, typically 0-20 bytes
Subject Unique Identifier	Varies, typically 0-20 bytes	Varies, typically 0-20 bytes
Extensions	50-300 bytes	50-300 bytes
Signature	384 bytes	64 bytes
Total	969 bytes – 1.432 bytes	319 bytes – 772 bytes

Attribute	RSA 3072-bit	ECDSA P-256
Generation Time	~271 milliseconds	~0.3 milliseconds
Environment	Intel i7, using OpenSSL, single-threaded operation.	
`openssl` command	openssl genpkey -algorithm RSA -out rsa_3072.key -pkeyopt rsa_keygen_bits:3072	openssl ecparam -genkey -name prime256v1 -out ecdsa_p256.key

Secure Device Identifiers

Increasing the Security of RSA and ECDSA



Algorithm	Key Size for 256-bit Security
RSA	~15360 bits (RSA 16384)
ECDSA	512 bits (P-521)

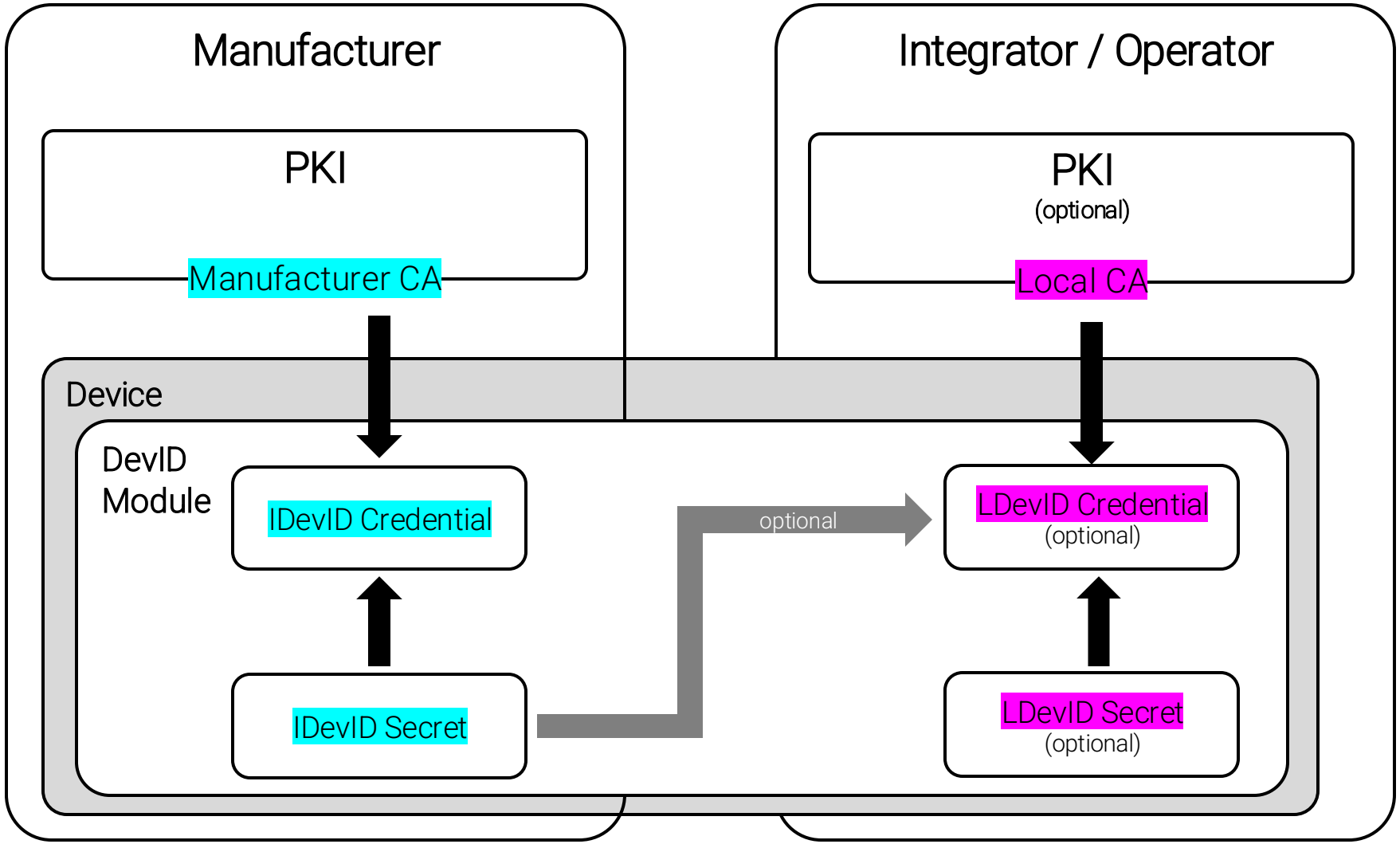
To double the security of ...

- ECDSA, you generally **double** the key size
- RSA, you'd generally need to **square** the key size

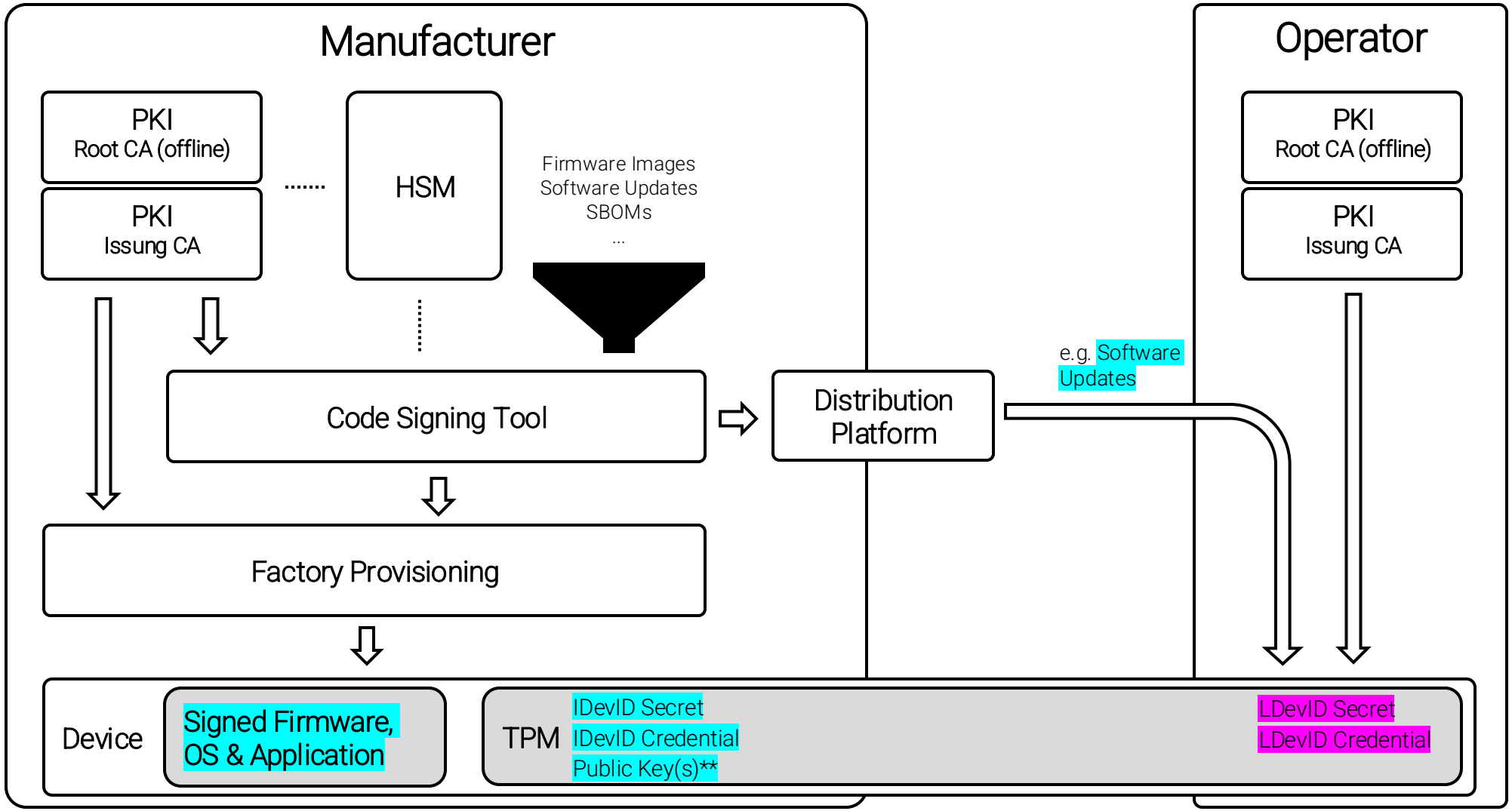
Attribute	RSA 16384	ECDSA P-521
Generation Time	~83228 milliseconds	~0.7 milliseconds
Environment	Intel i7, using OpenSSL, single-threaded operation.	
`openssl` command	openssl genpkey -algorithm RSA -out rsa_16384.key -pkeyopt rsa_keygen_bits:16384	openssl ecparam -genkey -name secp521r1 -out ecdsa_p521.key

Secure Device Identifiers

Trust hierachy



Simplified summary



* Infrastructure (PKI, HSM, Code Signing, etc.) depending on OEM
** Could also include third-party Public Keys

Appendix

Selected Root-of-Trust

... and their supported APIs

Root of Trust Type	PKCS#11	TSS (TPM Software Stack)	GlobalPlatform TEE API	OpenSSL Engine	Others
HSM	✓	✗	✗	✓	Microsoft CNG, Java JCE, OpenSC, Thales nShield Connect, Gemalto SafeNet Luna HSM Client YubiHSM2
TPM	✓ (via Middleware)	✓	✗	✓ (via TSS Integration)	Windows CNG, tpm2-tools, jTSS, ...
TEE	✗	✗	✓	✗	ARM Trustzone, OP-TEE, Intel SGX, ...
SE	✓ (via Middleware)	✗	✗	✗	Java Card, Global Platform Secure Element API, Android Keystore, ...

Selected Root-of-Trust

… and their supported functions

May be used together

Function	TPM	TEE	SE	HSM
Secure Boot and Trusted Boot	✔	✔	✘	✘
Cryptographic Key Management	✔	✔	✔	✔
Measurement and Attestation	✔	✔	✘ *	✘
Secure Storage	✔	✔	✔	✔
Encryption and Decryption	✔	✔	✔	✔
Digital Signatures and Verification	✔	✔	✔	✔
Authentication	✔	✔	✔	✔
Random Number Generation	✔	✔	✔	✔
Protection Against Physical Attacks	✔	✔	✔	✔
Certificate Management	✔	✘ **	✘ ***	✔
Secure Firmware Updates	✔	✔	✔	✘

* e.g.NXP's SE05x supports measurement and attestation

** e.g. ARM TrustZone supports certificate managmenent (generation, storage, usage)

*** e.g. STMicro STSAFE supports certificate management

Secure Update Methods

Method	Delivery Method	Management	Focus	Use Cases
Over-the-Air (OTA) Updates	Wireless (e.g., Wi-Fi, cellular, Bluetooth)	Typically device-initiated or triggered by a remote server	Wireless delivery directly to devices	Smartphones, IoT devices, automotive systems
Network-Based Updates	LAN/WAN, network-based delivery	Managed by enterprise IT, network administrators, or centralized servers	Consistent updates across multiple devices	Enterprise networks, server environments, large-scale deployments
Manual Updates	Physical media (e.g., USB drives), or manual download	User-initiated, often controlled by administrators or users	Offline updates, user-controlled process	Industrial equipment, offline devices, embedded systems, air-gapped networks

DevId Modules

Details

Functionality	Service Interface	Management Interface
Initialization (7.2.1)	✓	✗
Enumeration of DevID Public Keys (7.2.2)	✓	✓
Enumeration of DevID Certificates (7.2.3)	✓	✓
Enumeration of DevID Certificate Chains (7.2.4)	✓	✓
Signing (7.2.5)	✓	✗
DevID Certificate Enable/Disable (7.2.6)	✓	✓
DevID Key Enable/Disable (7.2.7)	✓	✓
LDevID Key Generation (7.2.8)	✓	✓
LDevID Key Insertion (7.2.9)	✓	✓
LDevID Key Deletion (7.2.10)	✓	✓
LDevID Certificate Insertion/Deletion (7.2.11-14)	✓	✓
Addition of RNG Entropy (7.2.15)	✓	✓
Operational Statistics Access	✗	✓
SMIv2 MIB Access (Clause 10)	✗	✓

Storage

- Secure Key Storage
- Certificate Storage
- Certificate Chain Storage

Asymmetric Cryptography Engine

- Key Pair Generation
- Digital Signature Operations
- Public/Private Key Handling

Random Number Generation (RNG) Engine

- RNG Entropy Source
- Deterministic/Non-deterministic RNG
- Integration with Cryptographic Operations

IEEE 802.1 AR - Secure Device Identifiers

IDecID and LDevID differences

Aspect	IDevID (Initial Device Identifier)	LDevID (Local Device Identifier)
Purpose	Permanent identity, typically assigned during manufacturing	Configurable identity, used for specific local network environments
Provisioning	Pre-provisioned during device manufacturing	Provisioned by the local network administrator or by the device itself
Uniqueness	Globally unique	Can be unique within a local context, not necessarily globally unique
Lifetime	Lifetime is typically the entire life of the device	Can be created, modified, or deleted as needed
Security Binding	Strongly bound to the device's hardware and cryptographic material	Can be bound to the device but more flexible in its binding process
Certificate Type	Usually comes with a manufacturer-provided X.509 certificate	Can be associated with a locally issued or self-signed certificate
Management	Typically managed by the manufacturer	Managed by the local network administrator
Authentication Use	Used for authenticating the device to external or global services	Used for authenticating the device in local or specific network environments
Revocation	Rarely revoked	Can be revoked or replaced by the local administrator
Key Management	Keys associated with IDevID are usually static and non-changeable	Keys can be changed or rotated as needed
Use Cases	Global identification, device attestation, secure boot	Localized authentication, role-based access control, network segmentation

Secure Device Identifiers

DevID Service Interface operations

Operation	Purpose	Required/Optional
Initialization (7.2.1)	Sets up the DevID module, preparing it for further operations.	Required
Enumeration of DevID Public Keys (7.2.2)	Lists all public keys associated with DevIDs stored in the module.	Required
Enumeration of DevID Certificates (7.2.3)	Lists all certificates associated with DevIDs stored in the module.	Required
Enumeration of DevID Certificate Chains (7.2.4)	Lists the certificate chains stored in the module.	Required
Signing (7.2.5)	Performs cryptographic signing using DevID keys stored in the module.	Required
DevID Certificate Enable/Disable (7.2.6)	Enables or disables specific DevID certificates.	Required
DevID Key Enable/Disable (7.2.7)	Enables or disables specific DevID keys.	Required
LDevID Key Generation (7.2.8)	Generates new LDevID keys within the DevID module.	Optional
LDevID Key Insertion (7.2.9)	Allows for the insertion of externally generated LDevID keys into the DevID module.	Optional
LDevID Key Delete (7.2.10)	Deletes LDevID keys from the DevID module.	Required (if LDevID Key Generation/Insertion is supported)
LDevID Certificate Insert (7.2.11)	Inserts LDevID certificates into the DevID module.	Required (if LDevIDs are supported)
LDevID Certificate Chain Insert (7.2.12)	Inserts LDevID certificate chains into the DevID module.	Required (if LDevIDs are supported)
LDevID Certificate Delete (7.2.13)	Deletes LDevID certificates from the DevID module.	Required (if LDevIDs are supported)
LDevID Certificate Chain Delete (7.2.14)	Deletes LDevID certificate chains from the DevID module.	Required (if LDevIDs are supported)
Addition of RNG Entropy (7.2.15)	Adds entropy to a deterministic Random Number Generator (RNG) within the module.	Required (if deterministic RNG is used)

Root of Trust

Examples and Type

Security Module	Examples	Type	Price Range
TPM	TPM 2.0, Intel PTT (Platform Trust Technology), AMD fTPM	**Internal**: Typically embedded in the motherboard or integrated into the processor.	1 – 20 € per unit
TEE	ARM TrustZone, Intel SGX (Software Guard Extensions), Qualcomm QSEE (Qualcomm Secure Execution Environment)	**Internal**: Embedded within the processor architecture, part of the system's main CPU.	0.50 – 10 € per device
SE	SIM cards, Apple Secure Enclave, Google Titan M, Java Card-based smart cards	**Both**: Can be internal (embedded in IoT gateways, metering devices) or external (SIM cards, smart cards).	0.50 – 5 € per unit
HSM	Thales nShield, AWS CloudHSM, YubiHSM2, Gemalto SafeNet Luna	**Both**: Can be external (networked HSM appliances, USB tokens) or internal (PCIe cards, embedded in specific devices like ATMs).	500 – 50,000+ € per unit

Thanks!

Florian Handke
Head of Industrial Security
Campus Schwarzwald
florian.handke@campus-schwarzwald.de